3D Gaussian Splatting for Real-Time Radiance Field Rendering

1. Introduction and Summary

3D Gaussian Splatting (3DGS) is a state-of-the-art technique for real-time Neural Rendering, introduced by Kerbl et al. (SIGGRAPH 2023). It addresses the limitations of both traditional geometry-based rendering and implicit neural representations (like NeRFs).

Core Concept

Instead of ray-marching a volumetric field (NeRF) or rendering a mesh (traditional), 3DGS represents the scene as a collection of **3D anisotropic Gaussians**. Each Gaussian has:

- **Position** (Mean μ)
- **Covariance** (Shape Σ)
- **Opacity** (α)
- **Color** (View-dependent Spherical Harmonics)

These Gaussians are "splatted" onto the screen, similar to point-based rendering, but with a mathematically rigorous projection that preserves their anisotropic shape.

2. Key Equations

2.1 The 3D Gaussian

A generic 3D Gaussian is defined by: $$ G(x) = e^{-\frac{1}{2} (x-\mu)^T \Sigma^{-1} (x-\mu)} $$

To ensure Σ is physically valid (positive semi-definite), it is parameterized by a scaling matrix S and rotation R: $$ \Sigma = R S S^T R^T $$

2.2 Projection (Splatting)

To render the Gaussian, we project it into 2D clip space. The 2D covariance Σ' is approximated by: $$ \Sigma' = J W \Sigma W^T J^T $$ where:

- W is the View Transformation Matrix.
- J is the Jacobian of the affine approximation of the perspective projection.

2.3 Volume Rendering

The final color of a pixel is computed by α-blending sorted Gaussians: $$ C = \sum_{i \in N} c_i \alpha_i \prod_{j=1}^{i-1} (1-\alpha_j) $$

3. Comparison with Traditional Methods

Feature	Traditional 3D (Rasterization)	PRT (Precomputed Radiance Transfer)	3D Gaussian Splatting
Geometry	Meshes (Triangles)	Static Meshes	Point Cloud of Gaussians
Rendering	Hardware Rasterization	Real-time Relighting Integration	Differentiable Rasterization (Broad Splats)
Pros	Mature, fast, hard surfaces	Good global illumination approx	Photorealistic, real-time, trains fast
Cons	Hard to model fuzzy/thin struct	Static geometry constraint	High memory usage (VRAM)

4. Basic Implementation in Elements

Below is a basic implementation of a 3DGS renderer using the **Elements** framework. We will:

1. Generate a cloud of random Gaussians.
2. Implement a shader that renders them as "Splats" (Billboards with Gaussian falloff).
3. Setup the Elements Scene Graph and Render Loop.

In [ ]:
```
import sys
import os
import pathlib
import numpy as np

print(f"Current working directory: {os.getcwd()}")

# --- SETTING UP PATHS FOR ELEMENTS ---
# We need to add the 'src' directory of the Elements project to sys.path
# Expected structure relative to this notebook: ../../../../src

project_root = pathlib.Path(os.getcwd()).parents[3] # go up 4 levels from ..
src_path = project_root / "src"

print(f"Project root (calculated): {project_root}")
print(f"Source path (calculated): {src_path}")

if src_path.exists():
    sys.path.insert(0, str(src_path))
    print("Added calculated src path to sys.path")
```

```
else:
    print(f"Warning: Calculated path {src_path} does not exist!")
    # Fallback: Absolute path based on user's known environment
    fallback_src = "/Users/papagian/GPcode/Elements/src"
    if os.path.exists(fallback_src):
        sys.path.insert(0, fallback_src)
        print(f"Used fallback absolute path: {fallback_src}")
    else:
        print("Critical: Could not find Elements source directory.")

# Try Import
try:
    import Elements
    print(f"Successfully imported Elements from: {Elements.__file__}")
except ImportError as e:
    print("Failed to import Elements. Please check your python environment a
    print(f"Current sys.path: {sys.path}")
    raise e

import Elements.pyECSS.math_utilities as util
from Elements.pyECSS.Entity import Entity
from Elements.pyECSS.Component import BasicTransform, RenderMesh
from Elements.pyGLV.GL.Scene import Scene
from Elements.pyGLV.GL.Shader import Shader, ShaderGLDecorator, InitGLShader
from Elements.pyGLV.GL.VertexArray import VertexArray
from Elements.pyGLV.GUI.ImguiDecorator import ImGUIecssDecorator2
from OpenGL.GL import GL_TRIANGLES, GL_BLEND, GL_ONE_MINUS_SRC_ALPHA, GL_SRC

print("All imports completed.")
```

In [ ]:

```
# -- Custom Shaders for Gaussian Splatting --

# Vertex Shader
# Projects the vertex (billboard corner) and passes properties to fragment s
vertex_shader_source = """
#version 410

layout (location = 0) in vec3 aPos;
layout (location = 1) in vec4 aColor;
layout (location = 2) in vec2 aUV;

uniform mat4 modelViewProj;

out vec4 vColor;
out vec2 vUV;

void main() {
    gl_Position = modelViewProj * vec4(aPos, 1.0);
    vColor = aColor;
    vUV = aUV;
}
"""

# Fragment Shader
# Computes the Gaussian Intensity alpha = exp(-0.5 * r^2)
fragment_shader_source = """
```

```
#version 410

in vec4 vColor;
in vec2 vUV;
out vec4 FragColor;

void main() {
    // Calculate distance from center of splat in UV space (-1 to 1)
    float r2 = dot(vUV, vUV);

    // Soft circle clip
    if (r2 > 1.0) discard;

    // Gaussian Falloff
    // We use a steeper falloff (multiply by 2 or 3) to make it look like a
    float alpha = exp(-2.0 * r2) * vColor.a;

    FragColor = vec4(vColor.rgb, alpha);
}
"""

print("Shaders defined.")
```

In [ ]:
```
class SimpleGaussian:
    def __init__(self, position, scale, color, opacity):
        self.position = np.array(position, dtype=np.float32)
        self.scale = scale # Simple scalar scale for this demo
        self.color = np.array(color, dtype=np.float32)
        self.opacity = opacity

def generate_random_gaussians(num=200):
    gaussians = []
    for _ in range(num):
        # Random position in a 4x4x4 cube
        pos = (np.random.rand(3) - 0.5) * 4.0
        # Random scale
        scale = np.random.rand() * 0.2 + 0.1
        # Random Color
        color = np.random.rand(3)
        # Random Opacity
        opacity = np.random.rand() * 0.8 + 0.2

        gaussians.append(SimpleGaussian(pos, scale, color, opacity))
    return gaussians

print("Data generation utils defined.")
```

In [ ]:
```
def run_3dgs_demo(max_frames=200):
    # 1. Initialize Scene
    print("Initializing Elements Scene...")
    scene = Scene()
    root = scene.world.createEntity(Entity(name="Root"))

    # 2. Setup Systems
    initUpdate = scene.world.createSystem(InitGLShaderSystem())
```

```
renderUpdate = scene.world.createSystem(RenderGLShaderSystem())

# 3. Create Splat Entity
splat_node = scene.world.createEntity(Entity(name="Splats"))
scene.world.addEntityChild(root, splat_node)

trans = scene.world.addComponent(splat_node, BasicTransform(name="SplatT
mesh = scene.world.addComponent(splat_node, RenderMesh(name="SplatMesh")

# 4. Generate Geometry (Billboards)
gaussians = generate_random_gaussians(100)

vertices = []
colors = []
uvs = []
indices = []
idx_counter = 0

# Create a quad for each Gaussian
for g in gaussians:
    s = g.scale
    # Vertices relative to center (Z=0 plane for billboard)
    v_pos = [
        g.position + np.array([-s, -s, 0]),
        g.position + np.array([ s, -s, 0]),
        g.position + np.array([ s,  s, 0]),
        g.position + np.array([-s,  s, 0]),
    ]
    v_uv = [
        np.array([-1.0, -1.0]),
        np.array([ 1.0, -1.0]),
        np.array([ 1.0,  1.0]),
        np.array([-1.0,  1.0])
    ]
    c = list(g.color) + [g.opacity]

    vertices.extend([v.astype(np.float32) for v in v_pos])
    colors.extend([np.array(c, dtype=np.float32)] * 4)
    uvs.extend([u.astype(np.float32) for u in v_uv])

    indices.extend([idx_counter, idx_counter+1, idx_counter+2, idx_count
    idx_counter += 4

# Add to Mesh
mesh.vertex_attributes.append(np.array(vertices, dtype=np.float32))
mesh.vertex_attributes.append(np.array(colors, dtype=np.float32))
mesh.vertex_attributes.append(np.array(uvs, dtype=np.float32))
mesh.vertex_index.append(np.array(indices, dtype=np.uint32))

# Attach VertexArray and Shader
scene.world.addComponent(splat_node, VertexArray(primitive=GL_TRIANGLES)
shaderDec = scene.world.addComponent(splat_node, ShaderGLDecorator(
    Shader(vertex_source=vertex_shader_source, fragment_source=fragment_
))

# 5. Build Scene Window
```

```
    # Initialize with ImGui support
    scene.init(windowWidth=1024, windowHeight=768, windowTitle="Elements: Ba

    # Enable Transparency
    glEnable(GL_BLEND)
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA)

    # Pre-render initialization
    scene.world.traverse_visit(initUpdate, scene.world.root)

    # Camera Setup
    eye = util.vec(0.0, 2.0, 10.0)
    target = util.vec(0.0, 0.0, 0.0)
    view = util.lookat(eye, target, util.vec(0.0, 1.0, 0.0))
    proj = util.perspective(60.0, 1024/768, 0.1, 100.0)

    # Render Loop
    print(f"Running render loop for {max_frames} frames...")
    running = True
    frame = 0
    while running and frame < max_frames:
        running = scene.render()
        scene.world.traverse_visit(renderUpdate, scene.world.root)

        # Simple rotation
        model = util.rotate(axis=(0,1,0), angle=frame)
        mvp = proj @ view @ model

        # Update Uniforms
        shaderDec.setUniformVariable(key='modelViewProj', value=mvp, mat4=Tr

        scene.render_post()
        frame += 1

    scene.shutdown()
    print("Finished.")

# Run
try:
    run_3dgs_demo()
except RuntimeError as e:
    print("Skipping execution (No OpenGL Context possible or other runtime e
```